

Computation Structures

Personal Notes

Mihir Rao

Contents

1	Introduction	2
1.1	Digital Systems and Abstraction	2
1.2	Digital Information	2
1.3	Unsigned Binary Numbers	3
1.4	Fixed Width and Modular Arithmetic	3
1.5	Two's Complement	4

1 Introduction

Computation Structures studies how layers of hardware and low-level software turn physical devices into programmable computer systems. Its approach is bottom-up: begin with binary signals and devices, combine them into combinational and sequential circuits, use those circuits to build a processor, and finally add the mechanisms needed by complete computer systems.

1.1 Digital Systems and Abstraction

Modern computers contain billions of devices, yet a designer rarely reasons about every device at once. Complexity is controlled through a hierarchy of *abstractions*. Each layer exposes a clean interface and hides the details of the layer below it. A logic gate, for example, can be described by how it maps input bits to an output bit even though different technologies may implement that behavior in entirely different ways.

The course develops four connected layers:

1. **Assembly language** provides the basic interface between software and processor hardware.
2. **Digital design** constructs combinational and sequential circuits from simpler components.
3. **Computer architecture** organizes those circuits into a programmable, general-purpose processor.
4. **Computer systems** adds mechanisms such as virtual memory and operating-system support so that the processor can be used safely and efficiently.

General-purpose processors are a useful focus because they are the basic building blocks of many systems and incorporate important techniques for efficient computation. The course uses RISC-V as its instruction-set architecture and Minispec as its hardware-description language. By separating the instruction interface from its circuit implementation, the same program can run on processors whose internal designs differ.

***Note:** Abstraction does not discard implementation details permanently. It lets a designer choose the right level of detail for a task, then move down a layer when performance, correctness, or physical constraints require it.*

1.2 Digital Information

An analog quantity can vary continuously. A digital system instead interprets information using a discrete collection of symbols called *digits*. A physical voltage is still continuous, but a circuit

may classify a range of low voltages as the digit 0 and a range of high voltages as the digit 1. This separation makes the representation tolerant of modest physical noise.

Binary representation uses the two digits 0 and 1, called *bits*. Two symbols are especially convenient for reliable physical implementation, and they support simple circuits for arithmetic, comparison, and Boolean operations.

1.3 Unsigned Binary Numbers

Definition 1.1 (Unsigned binary encoding). Let $b_{N-1} \cdots b_1 b_0$ be an N -bit string, with each $b_i \in \{0, 1\}$. Its unsigned value is

$$U(b) = \sum_{i=0}^{N-1} 2^i b_i.$$

The bit b_i therefore has positional weight 2^i .

The least significant bit b_0 has weight 1, while the most significant bit b_{N-1} has weight 2^{N-1} . The smallest unsigned value is represented by the all-zero string, and the largest by the all-one string. Consequently, an N -bit unsigned word represents exactly the range

$$0 \leq U(b) \leq 2^N - 1.$$

Example: The binary word 011111010000_2 represents

$$2^{10} + 2^9 + 2^8 + 2^7 + 2^6 + 2^4 = 2000.$$

Long binary strings are often written in hexadecimal, or base 16. Because $16 = 2^4$, every group of four bits corresponds to one hexadecimal digit: 0–9 followed by A–F. Grouping from the right gives

$$0111\ 1101\ 0000_2 = \mathbf{0x7D0}.$$

This notation changes only how the bits are displayed; it does not change the stored value.

1.4 Fixed Width and Modular Arithmetic

Binary addition and subtraction use the same carrying and borrowing ideas as decimal arithmetic. Hardware words, however, have a fixed width. An operation may mathematically produce a bit beyond the most significant stored position. When that extra bit is discarded, the result *overflows* the available word.

For unsigned N -bit arithmetic, discarding all bits beyond the word is equivalent to computing modulo 2^N :

$$\text{word}_N(a + b) = (a + b) \bmod 2^N.$$

The representable values therefore form a cycle. Adding 1 to the all-one word produces the all-zero word, just as a clock wraps around after completing one revolution. For example, in three bits,

$$011_2 - 101_2 \equiv 110_2 \pmod{2^3}.$$

As an unsigned value, 110_2 means 6. The same bit pattern can instead mean -2 if the system adopts an appropriate signed encoding.

1.5 Two's Complement

A direct sign-and-magnitude scheme could reserve the most significant bit for the sign and use the remaining bits for magnitude. That choice has two representations of zero and requires separate rules for addition and subtraction. Modern processors instead usually use two's complement, which is aligned with fixed-width modular arithmetic.

Definition 1.2 (Two's-complement encoding). The signed value of an N -bit word $b_{N-1} \cdots b_0$ is

$$S(b) = -2^{N-1}b_{N-1} + \sum_{i=0}^{N-2} 2^i b_i.$$

The most significant bit has negative weight; all other bits retain their unsigned weights.

Words whose most significant bit is 0 have the same nonnegative value under the unsigned and two's-complement interpretations. Words whose most significant bit is 1 represent negative values. The available range is

$$-2^{N-1} \leq S(b) \leq 2^{N-1} - 1.$$

Thus $10 \cdots 0$ is the most negative value, $01 \cdots 1$ is the most positive, and $11 \cdots 1$ represents -1 . There remains one representation of zero.

Two's complement also makes negation inexpensive. If $\tilde{A}$ denotes the result of inverting every bit of an N -bit word A , then

$$\tilde{A} = (2^N - 1) - A \equiv -1 - A \pmod{2^N}.$$

Adding one therefore gives

$$-A \equiv \tilde{A} + 1 \pmod{2^N}.$$

Subtraction can now be performed by the same circuit used for addition: $A - B = A + (-B)$. For a four-bit example,

$$\begin{aligned} -6 : \quad 0110_2 &\longrightarrow 1001_2 + 1 = 1010_2, \\ 3 - 6 : \quad 0011_2 + 1010_2 &= 1101_2 = -3. \end{aligned}$$

The arithmetic is ordinary addition of bit patterns; the signed interpretation of the result comes from the weight assigned to the highest-order bit.

Note: A bit pattern has no inherent numerical meaning. Width and encoding determine its interpretation. For example, the four-bit word 1101_2 is unsigned 13 but two's-complement -3 .

Binary digits provide a robust representation of information, fixed-width arithmetic naturally behaves modulo a power of two, and two's complement uses that modular structure to support positive and negative arithmetic with the same basic hardware.

Source: Daniel Sanchez and Silvina Hanono Wachman, MIT 6.004 Computation Structures, Fall 2020, Lecture 1, "Introduction," notes by Andrew Lin. Official course site <https://6.004.mit.edu> and MIT-hosted student notes https://6191.mit.edu/_static/fall24/resources/references/6004_student_notes_FA20.pdf.